

SQL Technique Review

Business Problem & Solution Reference — HealthcarePractice Portfolio
Charles Richardson | Compiled Week 5, July 2026

This document collects the SQL techniques rotated through recent portfolio weeks, each framed as a real business question against the HealthcarePractice schema rather than an abstract syntax exercise. Every query has been run and verified against the actual dataset — the results shown are real, not illustrative.

Two techniques get extra depth: the existence pattern (**NOT EXISTS** / **LEFT JOIN + IS NULL**) and the self-join, both flagged as needing dedicated review time separate from the day-to-day portfolio pace.

Contents

1. Window Functions — RANK() vs. DENSE_RANK()
2. Existence Pattern — NOT EXISTS
3. Existence Pattern — LEFT JOIN + IS NULL (the alternate form)
4. Self-Join — Hierarchy / Reporting Structure
5. Self-Join — Near-Duplicate Detection
6. Aggregation & Cross-Environment Validation
7. CTE + LAG() — Trend Analysis

1. Window Functions

RANK() OVER (PARTITION BY ...)

Business Problem

The network operations team wants to rank each provider's contracts by **rate_modifier**, so the highest-rate contract per provider is easy to spot at a glance. Ranking has to reset for every new provider — the numbers shouldn't run continuously across the whole table.

Solution

```
SELECT
  contract_id,
  provider_id,
  contract_type,
  rate_modifier,
  RANK() OVER (
    PARTITION BY provider_id
    ORDER BY rate_modifier DESC
  ) AS rate_modifier_rank
FROM network_contracts
ORDER BY provider_id, rate_modifier_rank;
```

PARTITION BY provider_id — restarts the ranking at 1 every time provider_id changes. Without this, the ranking would run straight through the whole table, comparing every contract against every other contract regardless of provider.

ORDER BY rate_modifier DESC — defines the ranking criteria, inside the OVER() clause specifically. This is separate from the outer query's ORDER BY, which just controls how results are displayed.

RANK() vs. DENSE_RANK()

rate_modifier	RANK()	DENSE_RANK()
1.20	1	1
1.20	1	1
1.15	3	2
1.10	4	3

RANK() leaves a gap after a tie — two rows tied for 1st means the next row jumps to 3rd, not 2nd.

DENSE_RANK() never skips — ties still share a rank, but the next distinct value always gets the very next integer. Memory aid: RANK() counts *rows*; DENSE_RANK() counts *distinct values*.

Verified Result

Run against the live network_contracts table: every row currently returns rank 1, because there is exactly one contract per provider in this dataset today. The query logic is correct — the data simply doesn't (yet) have a provider with multiple contracts to visibly demonstrate the ranking spread. A manufactured CTE with duplicate values is a good way to see RANK() vs. DENSE_RANK() diverge on demand.

Key Takeaway

PARTITION BY controls *which rows compete against each other*; ORDER BY inside OVER() controls *how they're ranked once grouped*. Getting the partition column wrong doesn't throw an error — it just silently ranks against the wrong comparison group, which is the kind of mistake that's easy to miss without checking the output against what you expect to see.

2. Existence Pattern

NOT EXISTS

Business Problem

Compliance needs a list of providers with **no currently active network contract** — either they were never contracted at all, or their contract has expired. This matters because `network_contracts` has **no status flag**; "active" has to be derived from a date range (`start_date` / `end_date`), not read off a column.

The Core Idea (Before Any SQL)

This is a "does a matching row exist, yes or no?" question. In plain English: *"Give me every provider where, if I go looking in `network_contracts` for a currently-active row tied to this provider, I find nothing."* That framing — not the syntax — is the part worth internalizing first.

Solution

```
SELECT
    provider_id,
    provider_name
FROM providers p
WHERE NOT EXISTS (
    SELECT 1
    FROM network_contracts nc
    WHERE nc.provider_id = p.provider_id
        AND (nc.end_date IS NULL
            OR nc.end_date >= CAST(GETDATE() AS date))
);
```

Line-by-Line

- **SELECT 1** inside the subquery — the actual value selected doesn't matter at all. NOT EXISTS only cares whether the subquery returns *any* row, not what's in it. SELECT 1 is a common convention signaling "I don't care about the data here, only whether it exists."
- **nc.provider_id = p.provider_id** — this correlates the subquery back to the outer row currently being evaluated. This is what makes it a *correlated* subquery rather than an independent one — it reruns conceptually for every row in the outer providers table.
- **nc.end_date IS NULL OR nc.end_date >= CAST(GETDATE() AS date)** — this is the "active" business rule, made explicit since there's no status column to check instead. NULL `end_date` means an open-ended, still-active contract; a future or today's `end_date` also still counts as active.

Verified Result

15 of 20 providers returned — including one provider never contracted at all, and a second whose contract expired 2025-12-31. Both correctly caught by the same query, confirming the date-range logic distinguishes "never contracted" from "contract lapsed" without needing two separate queries.

Key Takeaway

NOT EXISTS answers a yes/no question about an entire correlated match, not about a specific column value. The subquery's WHERE clause is where all the real logic lives — everything that defines "a matching row I care about" goes there, not in the outer query.

3. Existence Pattern — Alternate Form

LEFT JOIN + IS NULL

Same Business Problem, Different Mechanics

This solves the identical question as Section 2 — providers with no active contract — using a different SQL mechanism. Both are worth knowing, since interviewers sometimes ask for one specifically, or ask you to explain the difference.

Solution

```
SELECT
    p.provider_id,
    p.provider_name
FROM providers p
LEFT JOIN network_contracts nc
    ON p.provider_id = nc.provider_id
    AND (nc.end_date IS NULL
        OR nc.end_date >= CAST(GETDATE() AS date))
WHERE nc.provider_id IS NULL;
```

How This Differs From NOT EXISTS

- **LEFT JOIN** keeps every row from providers regardless of whether a match is found in network_contracts — that's the defining behavior of a LEFT JOIN.
- The active-contract condition moves **into the ON clause** here, not a separate WHERE — this matters. Putting it in WHERE instead would filter out the provider entirely on non-matching rows rather than just failing to match, since WHERE runs after the join has already happened.
- If a provider has no active contract, every network_contracts column comes back **NULL** for that row — so **WHERE nc.provider_id IS NULL** is the final filter that isolates exactly those "no match found" rows.

Key Takeaway

NOT EXISTS tends to read more directly as "does this exist, yes or no" and is often the better performer on large tables, since the engine can stop as soon as one match is found rather than materializing a full join. LEFT JOIN + IS NULL is more visually intuitive once you've seen the NULL-column behavior explained — picking one over the other is often a style/performance call, not a correctness one.

4. Self-Join

Hierarchy / Reporting Structure

Business Problem

A manager wants a report showing every employee alongside their manager's name, including the person at the very top who has no manager at all. This is a classic org-chart pattern — practiced here against a small throwaway table, since the claims schema has no natural hierarchy relationship to demonstrate it on.

Table Setup

```
CREATE TABLE dbo.employee_hierarchy (  
    employee_id INT NOT NULL,  
    employee_name VARCHAR(50) NOT NULL,  
    manager_id INT NULL  
);
```

manager_id is nullable on purpose — the person at the top of the org has no manager, and that has to be representable.

Solution

```
SELECT  
    e.employee_name AS employee,  
    m.employee_name AS manager  
FROM dbo.employee_hierarchy e  
LEFT JOIN dbo.employee_hierarchy m  
    ON e.manager_id = m.employee_id  
ORDER BY m.employee_name, e.employee_name;
```

Line-by-Line

- The **same table is joined to itself**, aliased twice (e for employee, m for manager) — this is what makes it a self-join rather than a normal join between two different tables.
- **e.manager_id = m.employee_id** — the join condition connects each employee's manager_id to the row where that same ID appears as someone else's employee_id.
- **LEFT JOIN**, not INNER JOIN — required so the top-of-hierarchy person (whose manager_id is NULL) still appears in the results, just with a blank manager column, instead of being silently dropped.

Verified Result

10 rows returned correctly — the top-level employee with a NULL/blank manager, and every other employee correctly paired with their direct manager's name, including a third-level relationship (an employee reporting to a manager who themselves reports to someone else).

Key Takeaway

A self-join is just a regular join where both sides happen to be the same table — the aliasing is what makes SQL treat them as two logically separate row sets. The LEFT JOIN vs. INNER JOIN choice is what determines whether the top of a hierarchy (or any row with no match on the self-referencing key) survives into the output.

5. Self-Join

Near-Duplicate Detection

Business Problem

The claims audit team is concerned about possible duplicate billing — the same member being billed twice for what might be the same service. A flat GROUP BY / HAVING COUNT(*) > 1 check for exact duplicates already returned zero rows, so the real question is subtler: are there claims for the **same member, close together in time**, that deserve a closer look even if they aren't byte-for-byte identical?

Solution

```
SELECT
  c1.claim_id AS claim_id_1,
  c2.claim_id AS claim_id_2,
  c1.member_id,
  c1.service_date AS service_date_1,
  c2.service_date AS service_date_2,
  ABS(DATEDIFF(day, c1.service_date, c2.service_date)) AS days_apart,
  c1.provider_id AS provider_id_1,
  c2.provider_id AS provider_id_2,
  c1.billed_amount AS billed_amount_1,
  c2.billed_amount AS billed_amount_2
FROM claims c1
JOIN claims c2
  ON c1.member_id = c2.member_id
  AND c1.claim_id < c2.claim_id
  AND ABS(DATEDIFF(day, c1.service_date, c2.service_date)) <= 7
ORDER BY c1.member_id, days_apart;
```

Line-by-Line

- **c1.member_id = c2.member_id** — the core self-join condition, pairing every claim with every other claim belonging to the same member.
- **c1.claim_id < c2.claim_id** — critical detail. Without this, every pair would show up twice (A-B and B-A), and every claim would also pair with itself. The strict less-than forces each unique pair to appear exactly once.
- **ABS(DATEDIFF(day, ...)) <= 7** — the actual business filter: only flag pairs within a 7-day window. ABS() is needed because DATEDIFF's sign depends on which date comes first, and the pair could occur in either order.

Verified Result

77 same-member pairs within 7 days (~14% of all claims) — but only 5 of those also shared a provider, and none of the 5 showed matching or near-identical billed amounts. No true duplicates found; the query functions as a triage/screening tool, narrowing thousands of possible comparisons down to a handful worth a human's attention, not a final verdict on its own.

Key Takeaway

The `claim_id < claim_id` trick generalizes to any self-join meant to compare "every row against every other row exactly once" — it's worth having as a reflex any time a self-join is producing double-counted or self-paired results.

6. Aggregation & Validation

Cross-Environment Row Count Check

Business Problem

After a pipeline loads data from a source system into a downstream copy (Azure SQL → Fabric Lakehouse, in this case), how do you confirm the load was complete and correct — not silently short a few rows, or duplicated?

Solution

```
SELECT 'Warehouse' AS source, COUNT(*) AS row_count
FROM HealthcarePractice_Warehouse.dbo.claims

UNION ALL

SELECT 'Lakehouse' AS source, COUNT(*) AS row_count
FROM HealthcarePractice_Lakehouse.dbo.claims;
```

Line-by-Line

- **UNION ALL**, not UNION — this simply stacks both result sets into one output; there's no need for UNION's automatic duplicate removal here since the two rows represent genuinely different sources, not duplicate data.
- A literal string ('Warehouse', 'Lakehouse') is selected alongside COUNT(*) purely as a label, so the two totals are distinguishable once stacked together in one result set.
- If cross-database querying isn't available in a given SSMS session/connection, the same check can be run as two separate single-table COUNT(*) queries and compared by eye.

Key Takeaway

This is a genuinely common real-world check — this portfolio's own Week 5 Thursday work found a live example of exactly this kind of mismatch, when a routine comparison surfaced four duplicate rows in one environment's copy of claims that didn't exist in the source. A simple row-count comparison is often the first thing worth running after any pipeline execution, before trusting the destination data.

7. CTE + Window Function

LAG() for Trend Analysis

Business Problem

Leadership wants to know whether denial reasons are trending up or down month over month — not just this month's raw count, but the change from the prior month for each denial reason.

Solution

```
WITH monthly_denials AS (  
    SELECT  
        denial_reason,  
        FORMAT(service_date, 'yyyy-MM') AS claim_month,  
        COUNT(*) AS denial_count  
    FROM claims  
    WHERE claim_status = 'Denied'  
    GROUP BY denial_reason, FORMAT(service_date, 'yyyy-MM')  
)  
SELECT  
    denial_reason,  
    claim_month,  
    denial_count,  
    LAG(denial_count) OVER (  
        PARTITION BY denial_reason  
        ORDER BY claim_month  
    ) AS prior_month_count,  
    denial_count - LAG(denial_count) OVER (  
        PARTITION BY denial_reason  
        ORDER BY claim_month  
    ) AS change_vs_prior_month  
FROM monthly_denials  
ORDER BY denial_reason, claim_month;
```

Line-by-Line

- The **CTE does the "dirty work"** — grouping and aggregating raw claims into one row per denial_reason/month combination. The outer query then does the analysis (LAG comparison) on top of already-clean, pre-aggregated data.
- **LAG(denial_count) OVER (PARTITION BY denial_reason ORDER BY claim_month)** — looks back one row within each denial_reason's own timeline, pulling the previous month's count into the current row.
- **LAG() returns NULL for the first row in each partition** — expected, not an error. The very first month for any denial reason has no "prior month" to compare against.

Key Takeaway

CTEs are a readability tool, not a performance optimizer — SQL Server inlines them before generating an execution plan, so a CTE and the equivalent flat query typically compile to the same plan. The value here is entirely about separating "prepare the data" from "analyze the data" so a complex query stays self-documenting.